Why look at the heap?

A few JDK tools and one library show what is on the heap of a running JVM.

- Garbage is invisible in the code, but obvious in a class histogram.
- Fewer and smaller objects mean fewer GCs and more headroom for a spike.
- `jps` finds the JVM; `jmap` and `jcmd` look inside it; `jstat` watches its GCs.
- JOL, a library, measures single objects; garbage per call (JMH) comes with the solutions.

`jps`: which JVM is that?

`jps` lists the running Java processes of the current user, so you can find the pid the other tools need.

```
$ jps -l
674  Sleep.java
2310 org.apache.maven.wrapper.MavenWrapperMain
2488 jdk.jcmd/sun.tools.jps.Jps
```

- Each line is a pid and a main class or JAR; `-l` prints the full name.
- `-v` adds the JVM flags, which shows whether compact object headers are on.

`jmap`: what is on the heap?

`jmap` attaches to a running JVM and prints its objects, class by class.

```
jmap -histo:live <pid>                          # reachable objects only (runs a full GC first)
jmap -histo <pid>                               # every object, garbage included
jmap -dump:live,format=b,file=heap.hprof <pid>   # full heap dump for an analyzer
```

- A histogram is cheap and small; a dump holds every object and field value.
- `:live` hides garbage, which is exactly what allocation tuning needs to see.

Reading a class histogram


```
 num     #instances         #bytes  class name (module)
-------------------------------------------------------
   1:         15430         698576  [B (java.base@25.0.4.1)
   2:          3302         399272  java.lang.Class (java.base@25.0.4.1)
   3:         15273         366552  java.lang.String (java.base@25.0.4.1)
```

- `#bytes` divided by `#instances` is the average object size: 24 B for `String`.
- Two histograms around a workload, subtracted, give the garbage it created.

`jcmd`: one tool, many commands

`jcmd` sends a diagnostic command to a running JVM; it covers most of what `jmap` and `jinfo` did.

```
jcmd                                # list the running JVMs and their pids
jcmd <pid> help                     # list the commands that JVM understands
jcmd <pid> GC.class_histogram       # the same histogram as jmap -histo:live
jcmd <pid> GC.class_histogram -all    # includes unreachable objects, no full GC
jcmd <pid> VM.flags                 # flags in effect, e.g. +UseCompactObjectHeaders
```

- `-all` counts garbage as well, so one call before and one after shows the allocation.
- `VM.flags` proves whether compact object headers were really on.

`jstat`: is the JVM busy collecting?

`jstat` samples the GC counters of a running JVM at an interval, without stopping it.

```
$ jstat -gcutil <pid> 1s 5               # five samples, one per second
     E       O    YGC     YGCT    FGC      GCT      (columns trimmed)
 32.79    2.42      7    0.078      0    0.078
  0.00    2.42      9    0.090      0    0.090
 65.57    2.42     10    0.095      0    0.095
 28.69    2.42     12    0.107      0    0.107
 95.08    2.42     13    0.113      0    0.113
```

- `E` and `O`: percent used of eden and old generation; eden fills up and empties again.
- `YGC` counts young GCs: here about 1.5 per second, all of them for garbage.

JOL: how big is one object?

Java Object Layout (JOL) is a small OpenJDK library that shows the header, fields, padding and size of real objects.

```
System.out.println(ClassLayout.parseInstance(Integer.valueOf(500)).toPrintable());

OFF  SZ   TYPE DESCRIPTION               VALUE
  0   8        (object header: mark)     0x0000000000000001 (non-biasable; age: 0)
  8   4        (object header: class)    0x0019e5d0
 12   4    int Integer.value             500
Instance size: 16 bytes
```

- A library, not a JDK tool: add `org.openjdk.jol:jol-core` to the build.

DEMO Demo: down to the basement

Find a running JVM and look at what it holds.

```
jps -l                                          # which JVM is that?
jcmd <pid> GC.class_histogram | head -12        # what is on its heap
jcmd <pid> VM.flags -all | grep UseCompactObjectHeaders
jstat -gcutil <pid> 1s 3                        # is it collecting right now?
```

- Look for `Integer`, `HashMap$Node`, `Object[]` and `$$Lambda` near the top.